



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

# Describing Meaning

## Week 6: Quantifiers and Scope

Dr. Thomas Stephen

School of Linguistic, Speech and Communication Sciences  
Trinity College Dublin

Semester 2 2026

Roadmap

# Today

- **Phenomenon:** sentences with more than one operator (quantifiers, negation, modals) can be scope ambiguous.

# Today

- **Phenomenon:** sentences with more than one operator (quantifiers, negation, modals) can be scope ambiguous.
- **Formal tools:** predicate-logic representations, with quantifiers

# Today

- **Phenomenon:** sentences with more than one operator (quantifiers, negation, modals) can be scope ambiguous.
- **Formal tools:** predicate-logic representations, with quantifiers
- **Cross-linguistic comparison:** languages vary in how flexible scope is (word order, scrambling, scope marking).

# Today

- **Phenomenon:** sentences with more than one operator (quantifiers, negation, modals) can be scope ambiguous.
- **Formal tools:** predicate-logic representations, with quantifiers
- **Cross-linguistic comparison:** languages vary in how flexible scope is (word order, scrambling, scope marking).

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)



# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers
- 5 Scope with negation

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers
- 5 Scope with negation
- 6 Linguistic evidence about scope

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers
- 5 Scope with negation
- 6 Linguistic evidence about scope
- 7 Neg-raising and modals

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers
- 5 Scope with negation
- 6 Linguistic evidence about scope
- 7 Neg-raising and modals
- 8 Three scope-takers

# Plan

- 1 Reminder: what  $\forall$  and  $\exists$  do
- 2 Two-place predicates (relations)
- 3 Structural ambiguity
- 4 Scope ambiguity with two quantifiers
- 5 Scope with negation
- 6 Linguistic evidence about scope
- 7 Neg-raising and modals
- 8 Three scope-takers

Reminder: quantifiers

Reminder: quantifiers



# Quantifiers as searching

- One way to think about  $\forall$  and  $\exists$  is as a systematic search through the relevant objects.

# Quantifiers as searching

- One way to think about  $\forall$  and  $\exists$  is as a systematic search through the relevant objects.
- $\exists x \varphi(x)$ : does the search find *at least one* object with property  $\varphi$ ?

# Quantifiers as searching

- One way to think about  $\forall$  and  $\exists$  is as a systematic search through the relevant objects.
- $\exists x \varphi(x)$ : does the search find *at least one* object with property  $\varphi$ ?
- $\forall x \varphi(x)$ : does the search confirm that *every* object has property  $\varphi$ ?

# Quantifiers as searching

- One way to think about  $\forall$  and  $\exists$  is as a systematic search through the relevant objects.
- $\exists x \varphi(x)$ : does the search find *at least one* object with property  $\varphi$ ?
- $\forall x \varphi(x)$ : does the search confirm that *every* object has property  $\varphi$ ?

# Search + sets example

- Domain:  $D = \{a, b, c\}$

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket \text{dog} \rrbracket = \{a, c\}$

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket \text{dog} \rrbracket = \{a, c\}$

## Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket dog \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket \neq \emptyset$ .



# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket dog \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket dog \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

$\forall x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket = D$ .

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket \text{dog} \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket \text{dog} \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

$\forall x \text{ dog}(x)$

True in this model iff  $\llbracket \text{dog} \rrbracket = D$ . Here:  $\{a, c\} \neq \{a, b, c\}$ , so **false**.

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket \text{dog} \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket \text{dog} \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

$\forall x \text{ dog}(x)$

True in this model iff  $\llbracket \text{dog} \rrbracket = D$ . Here:  $\{a, c\} \neq \{a, b, c\}$ , so **false**.

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket dog \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

$\forall x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket = D$ . Here:  $\{a, c\} \neq \{a, b, c\}$ , so **false**.

- $\exists$  succeeds once it finds a dog;  $\forall$  fails once it finds a non-dog (here:  $b$ ).

# Search + sets example

- Domain:  $D = \{a, b, c\}$
- Dogs:  $\llbracket dog \rrbracket = \{a, c\}$

$\exists x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket \neq \emptyset$ . Here:  $\{a, c\} \neq \emptyset$ , so **true**.

$\forall x \text{ dog}(x)$

True in this model iff  $\llbracket dog \rrbracket = D$ . Here:  $\{a, c\} \neq \{a, b, c\}$ , so **false**.

- $\exists$  succeeds once it finds a dog;  $\forall$  fails once it finds a non-dog (here:  $b$ ).

# Exercise

Keep the same domain  $D = \{a, b, c\}$ .

- 1 If  $\llbracket \text{dog} \rrbracket = \emptyset$ , what are the truth values of  $\exists x \text{ dog}(x)$  and  $\forall x \text{ dog}(x)$ ?

# Exercise

Keep the same domain  $D = \{a, b, c\}$ .

- 1 If  $\llbracket \text{dog} \rrbracket = \emptyset$ , what are the truth values of  $\exists x \text{ dog}(x)$  and  $\forall x \text{ dog}(x)$ ?
- 2 If  $\llbracket \text{dog} \rrbracket = D$ , what are the truth values of  $\exists x \text{ dog}(x)$  and  $\forall x \text{ dog}(x)$ ?



# Exercise

Keep the same domain  $D = \{a, b, c\}$ .

- ① If  $\llbracket \text{dog} \rrbracket = \emptyset$ , what are the truth values of  $\exists x \text{ dog}(x)$  and  $\forall x \text{ dog}(x)$ ?
- ② If  $\llbracket \text{dog} \rrbracket = D$ , what are the truth values of  $\exists x \text{ dog}(x)$  and  $\forall x \text{ dog}(x)$ ?

# Restriction

- Natural language quantification is usually restricted.

# Restriction

- Natural language quantification is usually restricted.
- *Every villager smiled* does not quantify over everything, only villagers.

# Restriction

- Natural language quantification is usually restricted.
- *Every villager smiled* does not quantify over everything, only villagers.
- In predicate logic we do this with implication and conjunction.

# Restriction

- Natural language quantification is usually restricted.
- *Every villager smiled* does not quantify over everything, only villagers.
- In predicate logic we do this with implication and conjunction.

# Easy examples

Let *villager*(*x*), *smile*(*x*).

- *Every villager smiled.*

# Easy examples

Let *villager*(*x*), *smile*(*x*).

- *Every villager smiled.*

$$\forall x(\textit{villager}(x) \rightarrow \textit{smile}(x))$$

# Easy examples

Let  $villager(x)$ ,  $smile(x)$ .

- *Every villager smiled.*

$$\forall x (villager(x) \rightarrow smile(x))$$

- *Some villager smiled.*



# Easy examples

Let  $villager(x)$ ,  $smile(x)$ .

- *Every villager smiled.*

$$\forall x (villager(x) \rightarrow smile(x))$$

- *Some villager smiled.*

$$\exists x (villager(x) \wedge smile(x))$$

# Easy examples

Let  $villager(x)$ ,  $smile(x)$ .

- *Every villager smiled.*

$$\forall x (villager(x) \rightarrow smile(x))$$

- *Some villager smiled.*

$$\exists x (villager(x) \wedge smile(x))$$

- *No villager smiled.*

# Easy examples

Let *villager*(*x*), *smile*(*x*).

- *Every villager smiled.*

$$\forall x(\textit{villager}(x) \rightarrow \textit{smile}(x))$$

- *Some villager smiled.*

$$\exists x(\textit{villager}(x) \wedge \textit{smile}(x))$$

- *No villager smiled.*

$$\neg \exists x(\textit{villager}(x) \wedge \textit{smile}(x))$$

# Exercise

Translate into predicate logic.

- 1 *Every dog barked.*

# Exercise

Translate into predicate logic.

- ① *Every dog barked.*
- ② *Some cat slept.*

# Exercise

Translate into predicate logic.

- 1 *Every dog barked.*
- 2 *Some cat slept.*
- 3 *No student laughed.*

# Exercise

Translate into predicate logic.

- 1 *Every dog barked.*
- 2 *Some cat slept.*
- 3 *No student laughed.*

# Set-theoretic cash-out

Let  $A = \llbracket villager \rrbracket$  and  $B = \llbracket smile \rrbracket$ .

- $\forall x (villager(x) \rightarrow smile(x))$  is true iff  $A \subseteq B$ .



# Set-theoretic cash-out

Let  $A = \llbracket villager \rrbracket$  and  $B = \llbracket smile \rrbracket$ .

- $\forall x(villager(x) \rightarrow smile(x))$  is true iff  $A \subseteq B$ .
- $\exists x(villager(x) \wedge smile(x))$  is true iff  $A \cap B \neq \emptyset$ .

# Set-theoretic cash-out

Let  $A = \llbracket \text{villager} \rrbracket$  and  $B = \llbracket \text{smile} \rrbracket$ .

- $\forall x(\text{villager}(x) \rightarrow \text{smile}(x))$  is true iff  $A \subseteq B$ .
- $\exists x(\text{villager}(x) \wedge \text{smile}(x))$  is true iff  $A \cap B \neq \emptyset$ .

## Two-place predicates

Two-place predicates

# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

$$\llbracket \textit{smile} \rrbracket = \{x : x \text{ smiles}\}$$

# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

$$\llbracket \textit{smile} \rrbracket = \{x : x \text{ smiles}\}$$

- Transitive predicate: a relation between individuals.

# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

$$\llbracket \textit{smile} \rrbracket = \{x : x \text{ smiles}\}$$

- Transitive predicate: a relation between individuals.

$$\llbracket \textit{love} \rrbracket = \{\langle x, y \rangle : x \text{ loves } y\}$$

# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

$$\llbracket \textit{smile} \rrbracket = \{x : x \text{ smiles}\}$$

- Transitive predicate: a relation between individuals.

$$\llbracket \textit{love} \rrbracket = \{\langle x, y \rangle : x \text{ loves } y\}$$

- In predicate logic:  $\textit{love}(x, y)$  is true iff  $\langle x, y \rangle \in \llbracket \textit{love} \rrbracket$ .



# Transitive verbs as relations

- Intransitive predicate: a set of individuals.

$$\llbracket \textit{smile} \rrbracket = \{x : x \text{ smiles}\}$$

- Transitive predicate: a relation between individuals.

$$\llbracket \textit{love} \rrbracket = \{\langle x, y \rangle : x \text{ loves } y\}$$

- In predicate logic:  $\textit{love}(x, y)$  is true iff  $\langle x, y \rangle \in \llbracket \textit{love} \rrbracket$ .

# Exercise

Translate into predicate logic. Use constants  $j$  for John. Use predicates  $dog(x)$ ,  $cat(x)$ ,  $happy(x)$ , and relations  $love(x, y)$ ,  $chase(x, y)$ .

- ① *Every dog is happy.*
- ② *John loves every dog.*
- ③ *Some dog chased some cat.*

# Answers

- 1 *Every dog is happy.*

# Answers

- ① *Every dog is happy.*

$$\forall x (dog(x) \rightarrow happy(x))$$

# Answers

- ① *Every dog is happy.*

$$\forall x (dog(x) \rightarrow happy(x))$$

- ② *John loves every dog.*

# Answers

- ① *Every dog is happy.*

$$\forall x (dog(x) \rightarrow happy(x))$$

- ② *John loves every dog.*

$$\forall x (dog(x) \rightarrow love(j, x))$$

# Answers

- ① *Every dog is happy.*

$$\forall x (dog(x) \rightarrow happy(x))$$

- ② *John loves every dog.*

$$\forall x (dog(x) \rightarrow love(j, x))$$

- ③ *Some dog chased some cat.*

# Answers

- ① *Every dog is happy.*

$$\forall x (dog(x) \rightarrow happy(x))$$

- ② *John loves every dog.*

$$\forall x (dog(x) \rightarrow love(j, x))$$

- ③ *Some dog chased some cat.*

$$\exists x (dog(x) \wedge \exists y (cat(y) \wedge chase(x, y)))$$



Structural ambiguity

# Structural ambiguity

- One string can have more than one syntactic structure.

# Structural ambiguity

- One string can have more than one syntactic structure.
- Different structures often correspond to different meanings.

# Structural ambiguity

- One string can have more than one syntactic structure.
- Different structures often correspond to different meanings.
- This is a useful comparison point for scope ambiguity.

# Structural ambiguity

- One string can have more than one syntactic structure.
- Different structures often correspond to different meanings.
- This is a useful comparison point for scope ambiguity.

# Example

*She saw the man with a telescope.*

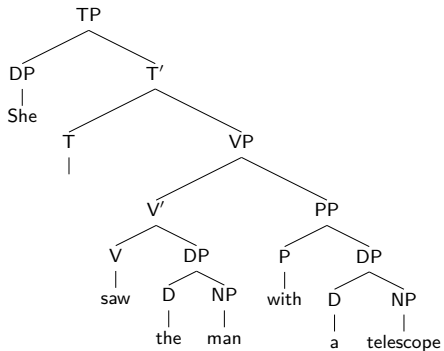
# Example

*She saw the man with a telescope.*

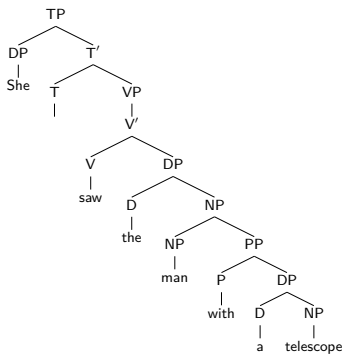
- Two readings:
  - instrument reading
  - modifier reading

# Trees for *She saw the man with a telescope*

## Instrument (PP attaches to VP)



## Modifier (PP attaches to NP)





## Scope with two quantifiers

Scope with two quantifiers

# Scope

- When a sentence contains more than one operator, there can be more than one way to build its truth conditions.

# Scope

- When a sentence contains more than one operator, there can be more than one way to build its truth conditions.
- Scope tracks which operator is evaluated “outside” which other operator.

# Scope

- When a sentence contains more than one operator, there can be more than one way to build its truth conditions.
- Scope tracks which operator is evaluated “outside” which other operator.

# Example

*Some number follows every number.*

# Example

*Some number follows every number.*

- What does this mean?

# Example

*Some number follows every number.*

- What does this mean?
- Is it:
  - for every number, there is a number exactly one greater than it,

# Example

*Some number follows every number.*

- What does this mean?
- Is it:
  - for every number, there is a number exactly one greater than it,
  - or there is a number that is the successor to every other number?



# Example

*Some number follows every number.*

- What does this mean?
- Is it:
  - for every number, there is a number exactly one greater than it,
  - or there is a number that is the successor to every other number?

## Two scope construals

Let  $number(x)$  and  $follows(x, y)$  mean “ $x$  follows  $y$ ”.

- **some** > **every**

## Two scope construals

Let  $number(x)$  and  $follows(x, y)$  mean “ $x$  follows  $y$ ”.

- **some** > **every**

$$\exists x \left( number(x) \wedge \forall y (number(y) \rightarrow follows(x, y)) \right)$$

## Two scope construals

Let  $number(x)$  and  $follows(x, y)$  mean “ $x$  follows  $y$ ”.

- **some** > **every**

$$\exists x \left( number(x) \wedge \forall y (number(y) \rightarrow follows(x, y)) \right)$$

- **every** > **some**

## Two scope construals

Let  $number(x)$  and  $follows(x, y)$  mean “ $x$  follows  $y$ ”.

- **some** > **every**

$$\exists x \left( number(x) \wedge \forall y (number(y) \rightarrow follows(x, y)) \right)$$

- **every** > **some**

$$\forall y \left( number(y) \rightarrow \exists x (number(x) \wedge follows(x, y)) \right)$$

# Example

*Every villager raised a dog.*

# Example

*Every villager raised a dog.*

- What readings does this sentence have?

# Example

*Every villager raised a dog.*

- What readings does this sentence have?
- Give a paraphrase for each reading.



# Example

*Every villager raised a dog.*

- What readings does this sentence have?
- Give a paraphrase for each reading.

## Two readings

- **Every** > **a**: for each villager, there exists (possibly different) dog they raised.

## Two readings

- **Every** > **a**: for each villager, there exists (possibly different) dog they raised.
- **A** > **every**: there exists a particular dog that every villager raised.

## Two readings

- **Every** > **a**: for each villager, there exists (possibly different) dog they raised.
- **A** > **every**: there exists a particular dog that every villager raised.

# Predicate-logic representations

Let  $villager(x)$ ,  $dog(y)$ ,  $raise(x, y)$ .

- **every** > **a**

# Predicate-logic representations

Let  $villager(x)$ ,  $dog(y)$ ,  $raise(x, y)$ .

- **every** > **a**

$$\forall x \left( villager(x) \rightarrow \exists y (dog(y) \wedge raise(x, y)) \right)$$

# Predicate-logic representations

Let *villager*(*x*), *dog*(*y*), *raise*(*x*, *y*).

- **every** > **a**

$$\forall x \left( \text{villager}(x) \rightarrow \exists y (\text{dog}(y) \wedge \text{raise}(x, y)) \right)$$

- **a** > **every**

# Predicate-logic representations

Let  $villager(x)$ ,  $dog(y)$ ,  $raise(x, y)$ .

- **every** > **a**

$$\forall x \left( villager(x) \rightarrow \exists y (dog(y) \wedge raise(x, y)) \right)$$

- **a** > **every**

$$\exists y \left( dog(y) \wedge \forall x (villager(x) \rightarrow raise(x, y)) \right)$$



# Exercise

For each sentence:

- list the scope-taking expressions,

# Exercise

For each sentence:

- list the scope-taking expressions,
- state the two readings you predict,

# Exercise

For each sentence:

- list the scope-taking expressions,
- state the two readings you predict,
- paraphrase both readings.

# Exercise

For each sentence:

- list the scope-taking expressions,
- state the two readings you predict,
- paraphrase both readings.

① *Every student read a paper.*

# Exercise

For each sentence:

- list the scope-taking expressions,
- state the two readings you predict,
- paraphrase both readings.

- 1 *Every student read a paper.*
- 2 *A teacher praised every student.*

# Exercise

For each sentence:

- list the scope-taking expressions,
- state the two readings you predict,
- paraphrase both readings.

- 1 *Every student read a paper.*
- 2 *A teacher praised every student.*

Scope with negation

Scope with negation

# Negation and a quantifier

*Not every student submitted a report.*



# Negation and a quantifier

*Not every student submitted a report.*

- What does this mean in plain terms?

# Negation and a quantifier

*Not every student submitted a report.*

- What does this mean in plain terms?
- Write it in predicate logic.

# Negation and a quantifier

*Not every student submitted a report.*

- What does this mean in plain terms?
- Write it in predicate logic.

# One standard representation

Let  $student(x)$ ,  $report(y)$ ,  $submitted(x, y)$ .

$$\neg \forall x \left( student(x) \rightarrow \exists y (report(y) \wedge submitted(x, y)) \right)$$

# Classic ambiguity

*Every student didn't pass.*

# Classic ambiguity

*Every student didn't pass.*

- What readings does this sentence have?

# Classic ambiguity

*Every student didn't pass.*

- What readings does this sentence have?

## Two logical possibilities

Let  $student(x)$  and  $pass(x)$ .

- **every** > **not**



## Two logical possibilities

Let  $student(x)$  and  $pass(x)$ .

- **every** > **not**

$$\forall x (student(x) \rightarrow \neg pass(x))$$

## Two logical possibilities

Let  $student(x)$  and  $pass(x)$ .

- **every** > **not**

$$\forall x (student(x) \rightarrow \neg pass(x))$$

- **not** > **every**

## Two logical possibilities

Let  $student(x)$  and  $pass(x)$ .

- **every** > **not**

$$\forall x (student(x) \rightarrow \neg pass(x))$$

- **not** > **every**

$$\neg \forall x (student(x) \rightarrow pass(x))$$

# Exercise

For this sentence, list the distinct readings you get and paraphrase each one. Give logical forms for each reading.

- 1 *A student didn't pass.*

# Exercise

For this sentence, list the distinct readings you get and paraphrase each one. Give logical forms for each reading.

- 1 *A student didn't pass.*

# Linguistic evidence about scope

Linguistic evidence about scope

# Why scope is a linguistic issue

- Not all readings are equally available.

# Why scope is a linguistic issue

- Not all readings are equally available.
- Accessibility depends on construction type, prosody, and language-specific morphosyntax.



# Why scope is a linguistic issue

- Not all readings are equally available.
- Accessibility depends on construction type, prosody, and language-specific morphosyntax.
- Cross-linguistic comparison helps separate:
  - what readings can be generated at all,

# Why scope is a linguistic issue

- Not all readings are equally available.
- Accessibility depends on construction type, prosody, and language-specific morphosyntax.
- Cross-linguistic comparison helps separate:
  - what readings can be generated at all,
  - what readings are easy to access, and

# Why scope is a linguistic issue

- Not all readings are equally available.
- Accessibility depends on construction type, prosody, and language-specific morphosyntax.
- Cross-linguistic comparison helps separate:
  - what readings can be generated at all,
  - what readings are easy to access, and
  - what grammatical cues bias one reading over another.

# Why scope is a linguistic issue

- Not all readings are equally available.
- Accessibility depends on construction type, prosody, and language-specific morphosyntax.
- Cross-linguistic comparison helps separate:
  - what readings can be generated at all,
  - what readings are easy to access, and
  - what grammatical cues bias one reading over another.

## German (scrambling and scope)

*weil der freundliche Museumsdirektor mindestens einer Frau fast jedes Gemälde gezeigt hat*  
because the friendly curator.NOM at-least one woman.DAT almost every painting.ACC shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

## German (scrambling and scope)

*weil der freundliche Museumsdirektor mindestens einer Frau fast jedes Gemälde gezeigt hat*  
because the friendly curator.NOM at-least one woman.DAT almost every painting.ACC shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- *at least one woman > almost every painting.*

## German (scrambling and scope)

*weil der freundliche Museumsdirektor mindestens einer Frau fast jedes Gemälde gezeigt hat*  
because the friendly curator.NOM at-least one woman.DAT almost every painting.ACC shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- *at least one woman* > *almost every painting*.
- inverse scope is unavailable or strongly dispreferred here.

## German (scrambling and scope)

*weil der freundliche Museumsdirektor mindestens einer Frau fast jedes Gemälde gezeigt hat*  
because the friendly curator.NOM at-least one woman.DAT almost every painting.ACC shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- *at least one woman* > *almost every painting*.
- inverse scope is unavailable or strongly dispreferred here.



## German (scrambling and scope)

*weil der freundliche Museumsdirektor mindestens einer Frau fast jedes Gemälde gezeigt hat*  
 because the friendly curator.NOM at-least one woman.DAT almost every painting.ACC shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- *at least one woman* > *almost every painting*.
- inverse scope is unavailable or strongly dispreferred here.

Let WOMAN(*x*), PAINTING(*x*), SHOW(*x*,*y*,*z*).

$$\exists x [\text{WOMAN}(x) \wedge \forall y (\text{PAINTING}(y) \rightarrow \text{SHOW}(d, y, x))]$$

$$\forall x [\text{PAINTING}(x) \rightarrow \exists y (\text{WOMAN}(y) \wedge \text{SHOW}(d, x, y))]$$

## German (scrambling and scope)

*weil mindestens einer Frau fast jedes Gemälde der freundliche Museumsdirektor  $t_1$   $t_2$  gezeigt hat*  
because at-least one woman.DAT almost every painting.ACC the friendly curator.NOM shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

## German (scrambling and scope)

*weil mindestens einer Frau fast jedes Gemälde der freundliche Museumsdirektor  $t_1$   $t_2$  gezeigt hat*  
 because at-least one woman.DAT almost every painting.ACC the friendly curator.NOM shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- both scope orders available under scrambling.

## German (scrambling and scope)

*weil mindestens einer Frau fast jedes Gemälde der freundliche Museumsdirektor  $t_1$   $t_2$  gezeigt hat*  
 because at-least one woman.DAT almost every painting.ACC the friendly curator.NOM shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- both scope orders available under scrambling.

# German (scrambling and scope)

*weil mindestens einer Frau fast jedes Gemälde der freundliche Museumsdirektor  $t_1$   $t_2$  gezeigt hat*  
 because at-least one woman.DAT almost every painting.ACC the friendly curator.NOM shown has  
*'because the friendly museum director showed almost every painting to at least one woman'*

- both scope orders available under scrambling.

Let WOMAN( $x$ ), PAINTING( $x$ ), SHOW( $x, y, z$ ).

$$\exists x [\text{WOMAN}(x) \wedge \forall y (\text{PAINTING}(y) \rightarrow \text{SHOW}(d, y, x))]$$

$$\forall x [\text{PAINTING}(x) \rightarrow \exists y (\text{WOMAN}(y) \wedge \text{SHOW}(d, x, y))]$$

## Japanese (scrambling and scope)

*Dareka-ga daremo-o aisite iru*  
 someone-NOM everyone-ACC love  
 'someone loves everyone'

## Japanese (scrambling and scope)

*Dareka-ga daremo-o aisite iru*  
 someone-NOM everyone-ACC love  
 'someone loves everyone'

- only *someone* > *everyone*.

## Japanese (scrambling and scope)

*Dareka-ga daremo-o aisite iru*  
 someone-NOM everyone-ACC love  
 'someone loves everyone'

- only *someone* > *everyone*.



# Japanese (scrambling and scope)

*Dareka-ga daremo-o aisite iru*  
someone-NOM everyone-ACC love  
'someone loves everyone'

- only *someone* > *everyone*.

Let,  $\text{LOVE}(x, y)$ .

$\exists x \forall y. \text{LOVE}(x, y)$

$\forall y \exists x. \text{LOVE}(x, y)$

## Japanese (scrambling and scope)

*Daremo-o [dareka-ga t aisite iru]*

everyone-ACC someone-NOM love

'everyone, someone loves' / 'for everyone, there is someone who loves them'

## Japanese (scrambling and scope)

*Daremo-o [dareka-ga t aisite iru]*

everyone-ACC someone-NOM love

'everyone, someone loves' / 'for everyone, there is someone who loves them'

- ambiguity becomes available, including *everyone* > *someone*.

## Japanese (scrambling and scope)

*Daremo-o [dareka-ga t aisite iru]*

everyone-ACC someone-NOM love

'everyone, someone loves' / 'for everyone, there is someone who loves them'

- ambiguity becomes available, including *everyone* > *someone*.

## Japanese (scrambling and scope)

*Daremo-o [dareka-ga t aisite iru]*

everyone-ACC someone-NOM love

'everyone, someone loves' / 'for everyone, there is someone who loves them'

- ambiguity becomes available, including *everyone* > *someone*.

Let, LOVE(*x*,*y*).

$\exists x \forall y. \text{LOVE}(x, y)$

$\forall y \exists x. \text{LOVE}(x, y)$

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu dōu yǒu yī-ge rén mǎi-le.*

every-1-CL book DOU have 1-CL person buy-LE

*'for every book, there is a person who bought it'*

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu dōu yǒu yi-ge ren mai-le.*

every-1-CL book DOU have 1-CL person buy-LE

*'for every book, there is a person who bought it'*

- universal wide.

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu dōu yǒu yi-ge ren mai-le.*

every-1-CL book DOU have 1-CL person buy-LE

*'for every book, there is a person who bought it'*

- universal wide.



# Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu dōu yǒu yi-ge ren mai-le.*

every-1-CL book DOU have 1-CL person buy-LE

*'for every book, there is a person who bought it'*

- universal wide.

Let  $\text{BOOK}(x)$ ,  $\text{PERSON}(x)$ ,  $\text{BUY}(x,y)$ .

$$\forall x [\text{BOOK}(x) \rightarrow \exists y (\text{PERSON}(y) \wedge \text{BUY}(y,x))]$$

$$\exists x [\text{PERSON}(x) \wedge \forall y (\text{BOOK}(y) \rightarrow \text{BUY}(x,y))]$$

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu yǒu yi-ge ren dōu mai-le.*

every-1-CL book have 1-CL person DOU buy-LE

*‘there is a person who bought every book’*

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu yǒu yī-ge rén dōu mǎi-le.*

every-1-CL book have 1-CL person DOU buy-LE

*'there is a person who bought every book'*

- existential wide.

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu yǒu yī-ge rén dōu mǎi-le.*

every-1-CL book have 1-CL person DOU buy-LE

*'there is a person who bought every book'*

- existential wide.

## Mandarin (*mei* and *dōu*)

*Mei-yi-ben shu yǒu yi-ge ren dōu mai-le.*

every-1-CL book have 1-CL person DOU buy-LE

*'there is a person who bought every book'*

- existential wide.

Let  $\text{BOOK}(x)$ ,  $\text{PERSON}(x)$ ,  $\text{BUY}(x,y)$ .

$$\forall x [\text{BOOK}(x) \rightarrow \exists y (\text{PERSON}(y) \wedge \text{BUY}(y,x))]$$

$$\exists x [\text{PERSON}(x) \wedge \forall y (\text{BOOK}(y) \rightarrow \text{BUY}(x,y))]$$

Neg-raising and modals

# Neg-Raising Predicates

For the sentence *Emma doesn't think that the dog is dangerous*:

- ① Give two paraphrases for two distinct readings.

# Neg-Raising Predicates

For the sentence *Emma doesn't think that the dog is dangerous*:

- ① Give two paraphrases for two distinct readings.
- ② In which reading does "think" take scope over negation and vice versa.



# Neg-Raising Predicates

For the sentence *Emma doesn't think that the dog is dangerous*:

- ① Give two paraphrases for two distinct readings.
- ② In which reading does "think" take scope over negation and vice versa.

Are there two distinct readings for *Emma doesn't know that the dog is dangerous*?

# A modals and negation

*John can't leave the prison.*

# A modals and negation

*John can't leave the prison.*

*John cannot leave the prison.*

# A modals and negation

*John can't leave the prison.*

*John cannot leave the prison.*

*John can not leave the prison.*

# A modals and negation

*John can't leave the prison.*

*John cannot leave the prison.*

*John can not leave the prison.*

- What are the two possible scope interactions between the modal "can" and negation?
- Which readings are available for which sentences?
- Is either reading more or less accisible in each case? Can you support the other one with enough context?

Summary

# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.

# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.
- Transitives denote relations; quantification can target either argument position.



# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.
- Transitives denote relations; quantification can target either argument position.
- Scope ambiguity arises when multiple operators interact.

# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.
- Transitives denote relations; quantification can target either argument position.
- Scope ambiguity arises when multiple operators interact.
- Cross-linguistic patterns suggest scope is sensitive to structural configuration and grammatical cues.

# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.
- Transitives denote relations; quantification can target either argument position.
- Scope ambiguity arises when multiple operators interact.
- Cross-linguistic patterns suggest scope is sensitive to structural configuration and grammatical cues.
- Neg-raising and modals provide further scope contrasts.

# Summary

- Quantifiers can be understood as “search procedures” over a domain, and as set-theoretic conditions.
- Transitives denote relations; quantification can target either argument position.
- Scope ambiguity arises when multiple operators interact.
- Cross-linguistic patterns suggest scope is sensitive to structural configuration and grammatical cues.
- Neg-raising and modals provide further scope contrasts.